

TECH

# TESTING

UNIT / INTEGRATION    E2E / PLAYWRIGHT

## UNIT / INTEGRATION

30 ВОПРОСОВ    7 ДНЕЙ    КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК    testing-library / jest / vitest

20 ВОПРОСОВ      7 ДНЕЙ

|                   |                                    |          |              |        |                                    |               |              |         |        |   |
|-------------------|------------------------------------|----------|--------------|--------|------------------------------------|---------------|--------------|---------|--------|---|
| ДЕНЬ 1<br>СРЕДНИЙ | ЧТО ТЕСТИРОВАТЬ                    |          | PYRAMID      | TROPHY | ЧТО ТЕСТИРОВАТЬ                    |               | PYRAMID      | TROPHY  |        |   |
|                   | Q1 · Q2 · Q3 · Q4 · Q5 · Q6        |          |              |        |                                    |               |              |         |        |   |
| ДЕНЬ 2<br>СРЕДНИЙ | PURE FN<br>HOOKS                   | REDUCERS | HOOKS        | ASYNC  | PURE FN<br>HOOKS                   | REDUCERS      | HOOKS        | ASYNC   |        |   |
|                   | Q7 · Q8 · Q9 · Q10                 |          |              |        |                                    |               |              |         |        |   |
| ДЕНЬ 3<br>СРЕДНИЙ | REACT COMPONENTS<br>IMPL    TESTID |          | USER-CENTRIC | NO     | REACT COMPONENTS<br>IMPL    TESTID |               | USER-CENTRIC | NO      |        |   |
|                   | Q11 · Q12 · Q13 · Q14              |          |              |        |                                    |               |              |         |        |   |
| ДЕНЬ 4<br>ТЯЖЁЛЫЙ | MOCK API<br>S                      | MSW      | TIME         | TIMERS | W                                  | MOCK API<br>S | MSW          | TIME    | TIMERS | W |
|                   | Q15 · Q16 · Q17 · Q18 · Q19        |          |              |        |                                    |               |              |         |        |   |
| ДЕНЬ 5<br>ТЯЖЁЛЫЙ | ABORT<br>MPTY                      | RACE     | LOADING      | ERROR  | E                                  | ABORT<br>MPTY | RACE         | LOADING | ERROR  | E |
|                   | Q20 · Q21 · Q22 · Q23 · Q24        |          |              |        |                                    |               |              |         |        |   |
| ДЕНЬ 6<br>СРЕДНИЙ | OPTIMISTIC<br>S    VALIDATION      | A11Y     | KEYBOARD     | FORM   | OPTIMISTIC<br>S    VALIDATION      | A11Y          | KEYBOARD     | FORM    |        |   |
|                   | Q25 · Q26 · Q27 · Q28 · Q29        |          |              |        |                                    |               |              |         |        |   |
| ДЕНЬ 7<br>СРЕДНИЙ | FLAKY                              |          |              |        | FLAKY                              |               |              |         |        |   |
|                   | Q30                                |          |              |        |                                    |               |              |         |        |   |

РИТМ    30-60 МИН ТЕОРИЯ    15-30 МИН КОД В КОНСОЛИ    15 МИН ОТВЕТЫ ВСЛУХ  
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ    ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

6 ВОПРОСОВ

**ЧТО ТЕСТИРОВАТЬ****PYRAMID****TROPHY**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ЧТО ТЕСТИРОВАТЬ

PYRAMID

TROPHY

ВОПРОСЫ ДНЯ

- Q01 Что тестировать unit-тестами?
- Q02 Что не стоит тестировать unit-тестами?
- Q03 Что тестировать integration-тестами?
- Q04 Что тестировать e2e-тестами?
- Q05 Как выбрать testing pyramid для frontend?
- Q06 Когда testing trophy лучше pyramid?

## Что тестировать unit-тестами?

### ОПИСАНИЕ

Pure functions, reducers, утилиты, custom hooks (изолированно), бизнес-логику. Маленькие, быстрые, изолированные. Покрывают edge cases.

---

### КАК РАБОТАЕТ

- 01 Pure functions.
- 02 Reducers.
- 03 Utility helpers.
- 04 Custom hooks isolated.
- 05 Edge cases coverage.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой проект.
- 02 Бизнес-логика.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Тесты implementation – хрупкие.
- 02 Coverage 100% без поведения.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем unit от integration отличается?
- 02 Что такое edge case?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/>

## ВОПРОС Q02

## Что не стоит тестировать unit-тестами?

### ОПИСАНИЕ

Frameworks, libraries (testing-library тестирует React, не Render). UI-сборку (для этого e2e). External services (mock через MSW). Trivial getters / setters.

### КАК РАБОТАЕТ

- 01 Framework / library code.
- 02 UI orchestration.
- 03 External services.
- 04 Trivial code.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review tests.

### ПОДВОДНЫЕ КАМНИ

- 01 Тесты на 'как jsx-рендерит' – пустые.
- 02 Coverage метрика над content.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое framework code?
- 02 Чем testing-trophy полезен?

### ПОЛНАЯ СТАТЬЯ

<https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications>

## Что тестировать integration-тестами?

### ОПИСАНИЕ

Component-flow: render + interaction + state-changes. Form submit + validation. Routing transitions. Data-flow между компонентами. Interaction со store.

---

### КАК РАБОТАЕТ

- 01 Component + interaction.
- 02 Form flow.
- 03 Routing.
- 04 Data flow.
- 05 Store integration.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой нетривиальный feature.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Слишком много integration без unit – медленно.
- 02 Без MSW – flaky.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое testing-trophy?
- 02 Чем integration от unit отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications>

## ВОПРОС Q04

## Что тестировать e2e-тестами?

### ОПИСАНИЕ

Critical user paths: login, signup, checkout, search-result. End-to-end intentions. Real browser, real APIs (или production-like staging). Smoke tests на каждый PR.

### КАК РАБОТАЕТ

- 01 Critical user paths.
- 02 Real browser.
- 03 Smoke per PR.
- 04 Full e2e nightly.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production-критичные.

### ПОДВОДНЫЕ КАМНИ

- 01 E2E на каждую фичу – slow.
- 02 Без isolation – flaky.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое critical path?
- 02 Чем smoke от full отличается?

### ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

## Как выбрать testing pyramid для frontend?

### ОПИСАНИЕ

Старая pyramid: много unit, меньше integration, мало e2e. Современная trophy (Kent Dodds): больше integration. Зависит от проекта.

---

### КАК РАБОТАЕТ

- 01 Pyramid: many unit, few e2e.
- 02 Trophy: many integration.
- 03 Adapt для проекта.
- 04 Coverage не равен качеству.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Test strategy.
- 02 Adoption.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Слепое следование pyramid.
- 02 100% unit без integration.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем trophy от pyramid отличается?
- 02 Что такое test-id?

---

### ПОЛНАЯ СТАТЬЯ

<https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications>



## ВОПРОС Q06

## Когда testing trophy лучше pyramid?

### ОПИСАНИЕ

В React-приложениях: integration-тесты testing-library проверяют близко к user behavior, дешевле e2e, но дают confidence лучше unit. Отсюда trophy с большим integration.

### КАК РАБОТАЕТ

- 01 React-heavy projects.
- 02 Behavior-testing focus.
- 03 Integration > unit для UI.
- 04 Cheaper than e2e.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 React apps.

### ПОДВОДНЫЕ КАМНИ

- 01 Не подходит для backend-lib.
- 02 Slow integration без правильного setup.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое behavior testing?
- 02 Чем testing-library полезен?

### ПОЛНАЯ СТАТЬЯ

<https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications>

**PURE FN****REDUCERS****HOOKS****ASYNC HOOKS****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

PURE FN

REDUCERS

HOOKS

ASYNC HOOKS

**ВОПРОСЫ ДНЯ**

Q07 Как тестировать pure functions?

Q08 Как тестировать reducers?

Q09 Как тестировать custom hooks?

Q10 Как тестировать async hooks?

## ВОПРОС Q07

## Как тестировать pure functions?

### ОПИСАНИЕ

Простые: input → assert output. Edge cases: empty, null, very large, very small.  
Property-based для random (fast-check). Без mocks – pure-functions deterministic.

---

### КАК РАБОТАЕТ

- 01 Input → output assertions.
- 02 Edge cases.
- 03 Property-based для random.
- 04 No mocks needed.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Utils, formatters, calculators.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Только happy path.
- 02 Без edge cases – пропуск bugs.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое property-based testing?
- 02 Чем fast-check полезен?

---

### ПОЛНАЯ СТАТЬЯ

<https://github.com/dubzzz/fast-check>

## Как тестировать reducers?

### ОПИСАНИЕ

Reducer = pure function (state, action) → state. Testable: dispatch action на initial state, assert new state. Покрыть все action types + edge cases.

---

### КАК РАБОТАЕТ

- 01 Pure function tests.
- 02 Initial state + action → new state.
- 03 All action types.
- 04 Edge cases.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Redux / useReducer.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Тесты на selector внутри reducer-теста.
- 02 Coverage метрика над behavior.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем createSlice проще тестировать?
- 02 Что такое reducer composition?

---

### ПОЛНАЯ СТАТЬЯ

<https://redux.js.org/usage/writing-tests>

## ВОПРОС Q09

## Как тестировать custom hooks?

### ОПИСАНИЕ

@testing-library/react renderHook для изоляции. Test return value + act() для interactions. Mock dependencies (router, fetch). Snapshot return value.

---

### КАК РАБОТАЕТ

- 01 renderHook + act().
- 02 Mock dependencies.
- 03 Test return value.
- 04 Test side-effects через mocks.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom hooks libraries.
- 02 Reusable logic.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Без act() – warnings + flaky.
- 02 Mock everything – false confidence.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое renderHook?
- 02 Чем act() полезен?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/react-testing-library/api/#renderhook>

## Как тестировать async hooks?

### ОПИСАНИЕ

renderHook + waitFor. Mock fetch / API. Test loading / success / error states. AbortController testing через cleanup. fake timers для timeouts.

---

### КАК РАБОТАЕТ

- 01 renderHook + waitFor.
- 02 Mock API через MSW / jest.fn.
- 03 Test states: loading/success/error.
- 04 Fake timers для timeouts.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 useFetch, useQuery, useDebounce.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Без waitFor – race-conditions.
- 02 Real fetch → flaky.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое waitFor?
- 02 Чем MSW от jest.fn полезнее?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/dom-testing-library/api-async/>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

REACT COMPONENTS

USER-CENTRIC

NO IMPL

TESTID

ЧТО УЧИМ В ЭТОТ ДЕНЬ

REACT COMPONENTS USER-CENTRIC NO IMPL TESTID

ВОПРОСЫ ДНЯ

Q11 Как тестировать React components?

Q12 Почему Testing Library советует тестировать как пользователь?

Q13 Почему не стоит тестировать implementation details?

Q14 Когда допустим data-testid?

## Как тестировать React components?

### ОПИСАНИЕ

render + getByRole / getByLabelText / getByText. user-event для interactions. assert на отображаемое UI, не на implementation. Mock side-effects.

---

### КАК РАБОТАЕТ

- 01 render + queries.
- 02 userEvent для actions.
- 03 Assert на UI, не на state.
- 04 Mock side-effects.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Components с поведением.

---

### ПОДВОДНЫЕ КАМНИ

- 01 data-testid вместо role.
- 02 Тесты implementation.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое user-event?
- 02 Чем getByRole от getByTestId лучше?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/react-testing-library/intro/>



## ВОПРОС Q12

## Почему Testing Library советует тестировать как пользователь?

### ОПИСАНИЕ

Тесты должны отражать как user взаимодействует с UI, не internal state. Получаем confidence в реальном поведении, тесты не ломаются на refactor implementation.

---

### КАК РАБОТАЕТ

- 01 User-centric тесты.
- 02 Не testing implementation.
- 03 Refactor-resistant.
- 04 Real confidence.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые UI-тесты.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Implementation tests ломаются на refactor.
- 02 Без user-centric – false confidence.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое behavior testing?
- 02 Чем user-event от fireEvent отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/guiding-principles/>

## Почему не стоит тестировать `implementation details`?

### ОПИСАНИЕ

Хрупкие тесты: refactor ломает тесты, хотя поведение то же. Code review показывает 'красные тесты' и нужно тратить время на исправление, не на ценность.

---

### КАК РАБОТАЕТ

- 01 Refactor-fragility.
- 02 Test-maintenance overhead.
- 03 False confidence (тесты red ≠ bug).
- 04 Slows refactoring.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review тестов.

---

### ПОДВОДНЫЕ КАМНИ

- 01 data-testid везде.
- 02 Тесты state instead of UI.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое implementation detail?
- 02 Чем behavior tests лучше?

---

### ПОЛНАЯ СТАТЬЯ

<https://kentcdodds.com/blog/testing-implementation-details>

## ВОПРОС Q14

**Когда допустим data-testid?****ОПИСАНИЕ**

Когда невозможно identify через role / label / text. Например, generic-обёртка без semantic. Использовать sparingly. Предпочитать accessible queries.

**КАК РАБОТАЕТ**

- 01 When no semantic.
- 02 Generic wrappers.
- 03 Use sparingly.
- 04 Prefer accessible queries.

**ГДЕ ВСТРЕЧАЕТСЯ**

- 01 Edge cases.
- 02 Legacy components.

**ПОДВОДНЫЕ КАМНИ**

- 01 data-testid везде – anti-pattern.
- 02 Без приоритета – каждый разработчик на своё усмотрение.

**МОГУТ ДОСПРОСИТЬ**

- 01 Что такое query priority?
- 02 Чем getByRole лучше?

**ПОЛНАЯ СТАТЬЯ**

<https://testing-library.com/docs/queries/about/#priority>

**MOCK API**

**MSW**

**TIME**

**TIMERS**

**WS**

#### ЧТО УЧИМ В ЭТОТ ДЕНЬ

MOCK API

MSW

TIME

TIMERS

WS

#### ВОПРОСЫ ДНЯ

Q15 Как мокать API?

Q16 Когда использовать MSW?

Q17 Как мокать time?

Q18 Как мокать timers?

Q19 Как мокать WebSocket?

## ВОПРОС Q15

## Как мокать API?

### ОПИСАНИЕ

MSW (Mock Service Worker) – лучший способ. Перехватывает fetch / XHR на network level. Тесты пишут как-будто real API. Reusable между unit / integration / e2e.

---

### КАК РАБОТАЕТ

- 01 MSW – network-level mocks.
- 02 Reusable across test types.
- 03 Realistic responses.
- 04 No code changes.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые API-зависимые тесты.

---

### ПОДВОДНЫЕ КАМНИ

- 01 jest.fn() per fetch – boilerplate.
- 02 Mock fetch globally – fragility.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое MSW?
- 02 Чем service-worker mock от jest.mock отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://mswjs.io/>

## Когда использовать MSW?

### ОПИСАНИЕ

Везде, где есть HTTP-вызовы: unit-hooks, integration, e2e. Один setup для всех уровней. Дополнительно – mocking WebSocket (MSW v1+).

---

### КАК РАБОТАЕТ

- 01 Unit / integration / e2e.
- 02 Single source of mocks.
- 03 WebSocket-mocking.
- 04 No global fetch override.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 React-приложения с API.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Mock в каждом тесте – duplicate.
- 02 Without MSW – flaky network tests.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое handlers?
- 02 Чем MSW от nock отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://mswjs.io/docs/getting-started>

## ВОПРОС Q17

## Как мокать time?

### ОПИСАНИЕ

jest fake-timers / vitest fake-timers. `jest.useFakeTimers()` →  
`jest.advanceTimersByTime(N)`. Mock `Date.now()` через `jest.setSystemTime()`.

### КАК РАБОТАЕТ

- 01 `jest.useFakeTimers()`.
- 02 `jest.advanceTimersByTime()`.
- 03 `jest.setSystemTime()` для `Date`.
- 04 `vi.useFakeTimers()` в `vitest`.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Animation tests.
- 02 Debounce/throttle.
- 03 Polling intervals.

### ПОДВОДНЫЕ КАМНИ

- 01 Real time → flaky.
- 02 Без advance – async tests hang.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое `systemTime`?
- 02 Чем `sinon-timers` от `jest` отличается?

### ПОЛНАЯ СТАТЬЯ

<https://jestjs.io/docs/timer-mocks>

## Как мокать timers?

### ОПИСАНИЕ

useFakeTimers replaces setTimeout/setInterval/etc. advanceTimersByTime / runAllTimers / runOnlyPendingTimers. Restore через useRealTimers.

---

### КАК РАБОТАЕТ

- 01 useFakeTimers.
- 02 advanceTimersByTime / runAllTimers.
- 03 Per-test useFakeTimers.
- 04 useRealTimers cleanup.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Debounce / throttle / interval tests.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Forget useRealTimers – leaks между tests.
- 02 runAllTimers может зацикливаться.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем runAllTimers от runOnlyPendingTimers отличается?
- 02 Что такое modern fake-timers?

---

### ПОЛНАЯ СТАТЬЯ

<https://jestjs.io/docs/timer-mocks>



## ВОПРОС Q19

## Как мокать WebSocket?

### ОПИСАНИЕ

MSW WebSocket handlers (v1+). Или mock-class implements WebSocket interface. Simulate onopen / onmessage / onclose. Use в integration-тестрах.

### КАК РАБОТАЕТ

- 01 MSW WS handlers.
- 02 Custom MockWebSocket class.
- 03 Simulate events.
- 04 Cleanup в afterEach.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 WS-heavy apps.

### ПОДВОДНЫЕ КАМНИ

- 01 Real WS в tests – flaky.
- 02 Без cleanup – leak подписок.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое MSW WS?
- 02 Чем sinon-websocket полезен?

### ПОЛНАЯ СТАТЬЯ

<https://mswjs.io/docs/api/ws>

**ABORT****RACE****LOADING****ERROR****EMPTY**

## ЧТО УЧИМ В ЭТОТ ДЕНЬ

ABORT

RACE

LOADING

ERROR

EMPTY

## ВОПРОСЫ ДНЯ

Q20 Как тестировать AbortController?

Q21 Как тестировать race conditions?

Q22 Как тестировать loading state?

Q23 Как тестировать error state?

Q24 Как тестировать empty state?

## ВОПРОС Q20

## Как тестировать AbortController?

### ОПИСАНИЕ

Mock fetch с возможностью abort. Trigger abort в test, assert `AbortError`. Test cleanup поведение (unmount → abort).

### КАК РАБОТАЕТ

- 01 Mock fetch с signal.
- 02 Trigger `ctrl.abort()`.
- 03 Assert `AbortError`.
- 04 Test unmount cleanup.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 `useFetch` с cancellation.
- 02 `Search-typeahead`.

### ПОДВОДНЫЕ КАМНИ

- 01 Real fetch не aborts в jsdom без mock.
- 02 Forget cleanup-test.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое `AbortError`?
- 02 Чем cleanup-тест полезен?

### ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

## Как тестировать race conditions?

### ОПИСАНИЕ

Trigger multiple async actions in sequence, control timing через fake-timers / mock-resolvers. Assert: только последний result применился.

---

### КАК РАБОТАЕТ

- 01 Multiple async actions.
- 02 Control timing с fake-timers.
- 03 Assert last-write-wins.
- 04 Use Promise.all + delayed resolvers.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Search.
- 02 Stale-response защита.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Real timing → flaky.
- 02 Без proper sequencing – false-positive.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое stale-response?
- 02 Чем fake-timers помогают?

---

### ПОЛНАЯ СТАТЬЯ

<https://jestjs.io/docs/timer-mocks>

## ВОПРОС Q22

## Как тестировать loading state?

### ОПИСАНИЕ

Mock API с delay (handler returns delayed Promise). Render → assert loading-indicator visible → waitFor → assert data-state.

### КАК РАБОТАЕТ

- 01 Mock с delayed Promise.
- 02 Assert loading visible.
- 03 waitFor finished state.
- 04 Test transitions.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Async loading components.
- 02 Suspense components.

### ПОДВОДНЫЕ КАМНИ

- 01 Real network – flaky.
- 02 Без delay – нельзя ассертировать loading.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое waitForElementToBeRemoved?
- 02 Чем delay в MSW полезен?

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/dom-testing-library/api-async/>

## Как тестировать error state?

### ОПИСАНИЕ

Mock API с error response. Assert error UI rendered. Test retry-flow если есть. Don't forget – onError side-effects (logging, etc).

---

### КАК РАБОТАЕТ

- 01 Mock error response.
- 02 Assert error UI.
- 03 Test retry-button.
- 04 Test side-effects (logging).

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Error-handling components.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Только happy path.
- 02 Без assertion на error UI.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое response.error?
- 02 Чем mock 500 от network-error отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://mswjs.io/docs/api/http>

## В О П Р О С Q 2 4

## Как тестировать empty state?

### ОПИСАНИЕ

Mock API с empty response. Assert empty-state UI visible (CTA, message). Test transitions empty → has-data.

### КАК РАБОТАЕТ

- 01 Mock empty response (e.g. []).
- 02 Assert empty-state UI.
- 03 Test transitions.
- 04 CTA navigation.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 List components.
- 02 Search results.

### ПОДВОДНЫЕ КАМНИ

- 01 Skip empty state – пропуск bug.
- 02 Тесты только на 1 row.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое CTA?
- 02 Зачем empty-state UX?

### ПОЛНАЯ СТАТЬЯ

<https://www.nngroup.com/articles/empty-state-interface-design/>

**OPTIMISTIC****A11Y****KEYBOARD****FORMS****VALIDATION**

## ЧТО УЧИМ В ЭТОТ ДЕНЬ

OPTIMISTIC

A11Y

KEYBOARD

FORMS

VALIDATION

## ВОПРОСЫ ДНЯ

Q25 Как тестировать optimistic update?

Q26 Как тестировать accessibility?

Q27 Как тестировать keyboard navigation?

Q28 Как тестировать forms?

Q29 Как тестировать validation?



## ВОПРОС Q25

## Как тестировать optimistic update?

### ОПИСАНИЕ

Mock mutation, trigger UI action, assert: optimistic UI immediately, then real response, assert: final UI matches. На failure: assert rollback.

---

### КАК РАБОТАЕТ

- 01 Trigger action.
- 02 Assert optimistic UI fast.
- 03 Wait для response.
- 04 Assert final state.
- 05 Test rollback on failure.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Likes / favorites / inline edit.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Real timing → flaky.
- 02 Без rollback test – пропуск bug.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое onMutate?
- 02 Чем react-query-test-utils полезен?

---

### ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

## Как тестировать accessibility?

### ОПИСАНИЕ

jest-axe – automated axe-core checks. expect(container).toHaveNoViolations().  
Storybook ally-addon. Manual через NVDA / VoiceOver.

---

### КАК РАБОТАЕТ

- 01 jest-axe в integration tests.
- 02 toHaveNoViolations matcher.
- 03 Storybook ally-addon.
- 04 Manual SR test.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 DS-компоненты.
- 02 Critical user paths.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Только automated – пропуск 70%.
- 02 Manual без структуры.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое axe?
- 02 Чем NVDA от VoiceOver отличается?

---

### ПОЛНАЯ СТАТЬЯ

<https://github.com/nickcolley/jest-axe>

## ВОПРОС Q 27

## Как тестировать keyboard navigation?

### ОПИСАНИЕ

user-event keyboard API: `keyboard('{Tab}')`, `keyboard('{ArrowDown}{Enter}')`. Assert focus moves correctly. Test focus-trap. Verify Esc/Enter behavior.

### КАК РАБОТАЕТ

- 01 `user.keyboard()`.
- 02 Tab/Arrow/Enter/Esc.
- 03 `Assert document.activeElement`.
- 04 Focus-trap test.

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Modals / dropdowns / forms.

### ПОДВОДНЫЕ КАМНИ

- 01 `fireEvent.keyDown` – не realistic.
- 02 Без focus-assertions – false confidence.

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое user-event keyboard?
- 02 Чем focus-trap полезен?

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/user-event/keyboard>

## Как тестировать forms?

### ОПИСАНИЕ

render → user.type() в каждое поле → user.click(submit) → assert: API called c correct data, или error UI shown. Cover validation rules.

---

### КАК РАБОТАЕТ

- 01 user.type per field.
- 02 user.click submit.
- 03 Mock API.
- 04 Assert payload или error.
- 05 Validation tests.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые формы.
- 02 Multi-step wizards.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Без user-event – нет realism.
- 02 Skip validation tests.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем user-event лучше fireEvent?
- 02 Что такое RHF tests?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/user-event/intro>

## В О П Р О С Q 2 9

## Как тестировать validation?

### ОПИСАНИЕ

Trigger invalid input, assert error message displays + input has aria-invalid. Cover schema-rules. Use Zod-schemas как single source of truth.

---

### КАК РАБОТАЕТ

- 01 Trigger invalid input.
- 02 Assert error UI + aria-invalid.
- 03 Test schema rules.
- 04 Zod-schema tests separately.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Any validated form.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Только submit-валидация.
- 02 Без accessibility-checks.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Чем Zod tests от UI tests отличаются?
- 02 Что такое aria-invalid?

---

### ПОЛНАЯ СТАТЬЯ

<https://zod.dev/>

# FLAKY

ЧТО УЧИМ В ЭТОТ ДЕНЬ

FLAKY

ВОПРОСЫ ДНЯ

Q30 Как избежать flaky tests?

## В О П Р О С Q 3 0

## Как избежать flaky tests?

### ОПИСАНИЕ

Deterministic mocks (MSW), fake-timers вместо real, await waitFor вместо setTimeout, isolated test data, no shared state, retry в CI с investigation, не игнорировать red tests.

---

### КАК РАБОТАЕТ

- 01 MSW + fake-timers.
- 02 waitFor вместо setTimeout.
- 03 Isolated data.
- 04 Retry с investigation.
- 05 Investigate flaky immediately.

---

### ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая test-suite.

---

### ПОДВОДНЫЕ КАМНИ

- 01 Hardcoded timeouts → flaky.
- 02 Shared state → race.
- 03 Retry-mask hides bugs.

---

### МОГУТ ДОСПРОСИТЬ

- 01 Что такое test isolation?
- 02 Чем waitFor от sleep лучше?

---

### ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/dom-testing-library/api-async/>

# СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [ ] Описать что unit / integration / e2e / pyramid vs trophy (Q1-Q6)
- ☐ [ ] Тестирование pure / reducers / hooks / async hooks (Q7-Q10)
- ☐ [ ] Тестирование components, user-centric, no implementation (Q11-Q14)
- ☐ [ ] Mock API / MSW / time / timers / WS (Q15-Q19)
- ☐ [ ] Тестирование AbortController / race / loading / error / empty (Q20-Q24)
- ☐ [ ] Тестирование optimistic / ally / keyboard / forms / validation (Q25-Q29)
- ☐ [ ] Избегать flaky tests (Q30)